Jaypee Institute of Information Technology, Noida

Department of Computer Applications

Project Title: Banking Credit Risk and Loan Portfolio Analytics

(A SQL based risk intelligence system)

Enrollment No.	**Name of Student**
992509260002	Rahul Bisht

Course Name: Data Management using SQL
Program: MSc in Data and Business Analytics
2nd Sem

2025 - 2026

TABLE OF CONTENTS

1. Project Overview 3
1.1 Problem Statement 3
1.2 Objectives 3
2. Dataset Overview 3
2.1 Table Structure 4
2.2 Schema Relationships 4
2.3 Intentional Data Quality Issues 3
3. System Architecture 4
3.1 System Flowchart 5
4. Implementation 6
Phase 1 — Data Audit & Cleaning 6
Phase 2 — PK / FK Constraint Check 7
Phase 3 — Multi-table JOINs & Master Portfolio Query 8
Phase 4 — Portfolio Quantization 9
Phase 5 — Liquidity Gap & DAR Analysis 10
Phase 6 — CTE + Window Functions 11
Phase 7 — Indexing & Performance Tuning 12
Phase 8a — Views & Stored Procedures 13
Phase 8b — What-If Interest Rate Simulation 14
5. Results Summary 16
6. Limitations & Future Scope 16
6.1 Limitations 16
6.2 Future Scope 17
7. Conclusion 18
Business Problems vs Solutions 18
8. References 18

1. PROJECT OVERVIEW

In the modern financial landscape, managing the balance between interest income and default risk is critical. This project utilizes a relational database to track the lifecycle of a loan from issuance to final payoff or default. Using advanced SQL techniques such as Common Table Expressions (CTEs), Window Functions, and Stored Procedures, the project segments the portfolio by risk level, analyzes the liquidity gap of overdue borrowers, and performs What-If simulations on interest rate volatility.

1.1 Problem Statement

The bank's leadership requires a transparent, real-time view of the loan portfolio's health. Fragmented reporting currently fails to link liquid account balances with outstanding loan statuses, creating three critical operational gaps:

#	Challenge	Description
1	Invisible Risk & Insolvent Borrowers	Difficulty identifying borrowers who lack liquid funds to repay overdue debt
2	Data Integrity Gaps	Dirty data (duplicates, null demographics, inconsistent dates) in raw transaction logs leads to skewed risk metrics
3	Revenue Uncertainty	No centralized logic engine to calculate Expected Credit Loss (ECL) vs. actual capital at risk

1.2 Objectives

- Data Sanitization & Engineering: SQL cleaning pipeline using COALESCE, CASE logic, and Regex to handle missing values and standardize the 50,000-row synthetic dataset.
- Portfolio Quantization: Aggregate total principal exposure and expected interest yield across all loan statuses (Active, Overdue, Paid Off).
- Liquidity & DAR Analysis: Calculate the Debt-to-Asset Ratio (DAR) to isolate high-risk Red Flag customers whose total debt exceeds their liquid assets.
- Stored Procedure Automation: Automated scripts to calculate dynamic Credit Health Scores based on transaction frequency and repayment history.
- Comparative Segment Analysis: Window Functions to rank default rates across customer segments to inform future credit policy.

2. DATASET OVERVIEW

The project uses a synthetic financial dataset of 50,000+ rows across 11 CSV tables, simulating a real-world banking system. The dataset was generated with intentional data quality issues (~7% noise) to reflect real-world ETL challenges.

2.1 Table Structure

Layer	Tables	Purpose
Core Entities	customers, accounts, loans, transactions	Primary transactional data
Dimension / Reference	customer_types, account_types, account_statuses, loan_statuses, transaction_types	Lookup and classification tables
Supporting	branches, addresses	Geographic and operational metadata

```
customer_types ⟶ customers ⟶ addresses
                     |
                     ↓
                  accounts ⟶ account_types
                     |        account_statuses
                     ↓
                   loans ⟶ loan_statuses
                     |
                  transactions ⟶ transaction_types
                               ⟶ branches
```

2.2 Schema Relationships

- customers (1:N) accounts
- customers (1:N) loans
- accounts (1:N) transactions
- branches (1:N) transactions
- All dimension tables linked via ID foreign keys

2.3 Intentional Data Quality Issues

Issue Type	Percentage	Description
Missing values (NULLs)	2%	NULL demographics in customers table
Typos in text fields	1%	Inconsistent name formatting
Duplicate rows	1%	500 duplicate TransactionIDs confirmed
Inconsistent formats	1%	Number and text format mismatches
Mixed date formats	1%	Non-standard date entries in loans
Non-sequential IDs	1%	Gap anomalies in primary keys
Future dates	1%	StartDate values beyond current date

3. SYSTEM ARCHITECTURE

The system is organized into four functional layers, each building on the previous to transform raw data into strategic intelligence.

Layer	Purpose	Key Deliverable
Data Input Layer	Raw data ingestion, PK/FK validation	Clean, trusted dataset
Processing Layer	Join engine, aggregation, risk calculation	Master portfolio query, DAR analysis
Optimization Layer	Query performance, indexing	Sub-second analytical queries
Output Layer	Reporting, automation, simulation	Views, Stored Procedures, What-If model

3.1 System Flowchart

The following flowchart illustrates the complete data flow and processing pipeline across all four layers of the system:

4. IMPLEMENTATION

Phase 1 — Data Audit & Cleaning

▸ The Big Picture First

Domain context: Before a bank runs any risk report, it needs to trust its data. Imagine calculating DAR on a customer whose balance shows NULL — the formula breaks. Or counting duplicate transactions as real money movement — the fraud detection gives wrong alerts. Data cleaning in banking is not optional. It's regulatory.

What Phase 1 does in one line:

> *We scanned 50,000+ rows across the customers and transactions tables, identified two types of corruption — disguised NULLs and duplicate records — and built two permanent clean tables that every subsequent phase depends on.*

▸ Key Findings

Audit Check	Result	Action Taken
NULL values in customers	0 true NULLs found	Checked for disguised NULLs (empty strings)
Empty strings in customers	44 customers affected	Converted using NULLIF(TRIM(col), '')
NULL values in loans	0 NULLs found	No action required
Duplicate TransactionIDs	500 duplicate rows confirmed	Deduplicated using ROW_NUMBER()
Future date anomalies	Present in StartDate	Flagged using CASE + CURDATE()

▸ SQL Concepts Applied

- NULLIF(TRIM(col), '') — converts empty strings to proper NULLs
- COALESCE(col, 'Unknown') — replaces NULLs with meaningful fallback values
- ROW_NUMBER() OVER (PARTITION BY TransactionID ORDER BY TransactionDate) — deduplication
- CASE WHEN StartDate > CURDATE() THEN 'Future Start - Anomaly' — date validation

▸ Key Insight

> *The NULL audit revealed zero true NULLs across both the customers and loans tables. However, a secondary audit using empty string detection uncovered 44 customers with missing name data disguised as empty strings — a data quality issue invisible to standard IS NULL checks and far more dangerous in production environments where NULL filtering is the default defensive measure.*

> *500 duplicate TransactionIDs — exactly 1% of 50,000 rows — were eliminated using the ROW_NUMBER() window function with PARTITION BY, preserving the earliest occurrence of each transaction as the canonical record. The resulting transactions_clean table serves as the trusted data layer for all downstream analytical phases.*

> *The final cleaning chain TRIM → NULLIF → COALESCE on the customers table standardizes whitespace, converts empty strings to proper NULLs, and replaces them with 'Unknown' — ensuring no NULL values propagate into downstream risk calculations. The resulting customers_final table is used exclusively in all post-cleaning analytical queries.*

> *CASE logic applied to InterestRate values categorized the entire loan book into Low Risk (below 5%), Medium Risk (5-10%), and High Risk (above 10%) segments — mirroring standard regulatory bucketing used in Basel III capital reporting frameworks.*

> *Date anomaly detection using CURDATE() identified loans with future start dates and logically impossible end-before-start configurations — both of which would corrupt loan lifecycle calculations including days overdue, maturity alerts, and active portfolio sizing.*

Phase 2 — PK / FK Constraint Check

▸ **The Big Picture First**

Domain context: A bank's database has thousands of accounts, loans, and transactions all referencing each other through IDs. A foreign key is a promise — "this AccountID in the loans table definitely exists in the accounts table." If that promise is broken, you have an **orphan record** — a loan that references an account that doesn't exist. Running a DAR analysis on orphan records gives you completely fictional risk numbers.

What Phase 2 does in one line:

> *Before running any JOIN-based analysis, we verified that every ID reference across all 6 key relationships in the schema is valid — confirming zero orphan records exist anywhere in the database.*

Relationship Checked	Orphan Count	Status
accounts → customers	0	Clean
loans → accounts	0	Clean
transactions → accounts	0	Clean
transactions → branches	0	Clean
loans → loan_statuses	0	Clean
customers → customer_types	0	Clean

▸ Key Insight

> *Referential integrity checks across all 6 key relationships returned zero violations, confirming the schema is relationally sound before proceeding to analytical querying.*

> *While the dataset contains intentional noise in the form of empty strings, duplicates, and date anomalies, the foreign key relationships between all core and dimension tables are fully intact — meaning the Phase 1 cleaning pipeline was sufficient to produce a trusted analytical data layer without any structural schema repair required.*

> *The LEFT JOIN + NULL filter pattern was applied consistently across all integrity checks. Unlike INNER JOIN which silently hides orphan records by excluding non-matching rows, LEFT JOIN deliberately exposes them — making it the correct diagnostic tool for referential integrity validation.*

> *A consolidated UNION ALL integrity summary confirmed all 6 relationships returned zero orphan records, providing a clean relational foundation for the multi-table JOIN engine in Phase 3.*

Phase 3 — Multi-table JOINs & Master Portfolio Query

▸ The Big Picture First

Domain context: The bank's data is split across 11 tables by design — this is called **normalization**. Customer names live in customers, account balances live in accounts, loan amounts live in loans. No single table has the full picture. Phase 3 is where you build the **JOIN engine** that stitches all these tables together into one unified view of the bank's loan book.

What Phase 3 does in one line:

> *We progressively connected all core tables starting from a simple 2-table JOIN all the way to a full 6-table master portfolio query that gives leadership a complete view of every customer, their account, their loan, and their loan status in a single result set.*

▸ JOIN Chain Built

- Step 3.1: customers + accounts + all dimension tables (INNER JOIN)
- Step 3.2: Extended with loans + loan_statuses
- Step 3.3: Full master query — all 6 tables unified in one result set
- Step 3.4: Enriched with transaction count and volume per account (LEFT JOIN + GROUP BY)
- Step 3.5: Customers with no loans identified (LEFT JOIN + NULL filter)

▸ Key Insight

> *The master portfolio query in Phase 3 successfully unified 6 tables — customers_final, customer_types, accounts, account_types, account_statuses, loans, and loan_statuses —*

> *into a single result set, providing a complete view of every borrower's identity, account profile, and loan status. This query serves as the relational backbone that every subsequent analytical phase is built upon.*

> *The LEFT JOIN pattern in Query 3.4 was used deliberately over INNER JOIN to preserve accounts with zero transaction history — ensuring that dormant accounts are not silently excluded from portfolio reporting, which would understate the bank's true customer base.*

> *Query 3.5 identified customers with accounts but no active loans using the LEFT JOIN + IS NULL pattern — producing a credit penetration list that represents untapped lending revenue from within the bank's existing verified customer base.*

Phase 4 — Portfolio Quantization

▸ The Big Picture First

Domain context: Now that the JOIN engine is built, leadership needs numbers. Not individual loan records — aggregated financial metrics. How much total money has the bank lent? How much does it expect to earn? Which loan status carries the most risk? This is called **portfolio quantization** — reducing 333 individual loan records into a handful of strategic financial figures that tell the complete story of the bank's lending book.

What Phase 4 does in one line:

> *We aggregated the entire loan book into total capital exposure, expected yield, and portfolio health metrics — broken down by loan status and customer segment — giving leadership a single-query financial scorecard.*

▸ SQL Concepts Applied

- SUM(PrincipalAmount) — total capital exposure per loan status
- SUM(PrincipalAmount * InterestRate) — expected yield calculation
- COUNT(LoanID) * 100.0 / SUM(COUNT(LoanID)) OVER () — portfolio % share using window function
- Multi-segment breakdown: Individual vs Small Business vs Large Enterprise

▸ Key Insight

> *Portfolio quantization revealed that Active loans dominate the book at 72.67% of total portfolio share ($12.6M principal), while Overdue loans represent 10.21% ($1.67M) — a Non-Performing Loan ratio that exceeds the standard 5% regulatory threshold and warrants provisioning action.*

> *Expected yield calculation using SUM(PrincipalAmount x InterestRate) produced a total portfolio yield of $1.55M at current interest rates. Of this, $136K is attributable to Overdue loans — revenue that is at risk of non-collection and must be treated as contingent income rather than confirmed revenue.*

> *The portfolio health scorecard used a nested window function — SUM(COUNT()) OVER() — to calculate each loan status's percentage share of the total portfolio in a single query, eliminating the need for subqueries or self-joins and demonstrating advanced SQL optimization.*

> *The isolated overdue exposure query confirmed $1.67M in principal at risk across 34 loans — this figure directly feeds the loan loss provisioning calculation and serves as the input for the ECL stress testing performed in Phase 8b.*

Phase 5 — Liquidity Gap & DAR Analysis

▸ The Big Picture First

Domain context: Knowing a loan is overdue is not enough. The real question is — does the borrower have any money left to repay it? A customer could have an overdue loan of $50,000 but $200,000 sitting in their account — they can repay, they're just late. But a customer with an overdue loan of $80,000 and only $2,000 in their account is genuinely insolvent. Phase 5 separates these two situations using the **Debt-to-Asset Ratio (DAR)** and produces the bank's official **Red Flag List**.

What Phase 5 does in one line:

> *We calculated every overdue borrower's total loan debt against their total liquid account balance, computed their DAR, and isolated the truly insolvent customers — those whose debt exceeds their assets — into an official Red Flag List ranked by uncollateralized debt.*

▸ DAR Formula

```
DAR = SUM(PrincipalAmount) / NULLIF(SUM(DISTINCT Balance), 0)
```

▸ Red Flag Criteria

- Loan status must be Overdue
- DAR must be greater than 1 (debt exceeds assets)
- UncollateralizedDebt = TotalLoanDebt - TotalLiquidAssets

▸ Key Insight

> *The Liquidity Gap analysis required SUM(DISTINCT a.Balance) rather than SUM(a.Balance) to correct for JOIN-induced row multiplication — a subtle but critical distinction where a single account balance is duplicated once per loan in the joined result set, causing a systematic overstatement of liquid assets without the DISTINCT qualifier.*

> *DAR calculation used NULLIF(SUM(DISTINCT Balance), 0) as divide-by-zero protection — converting zero-balance accounts to NULL before division, ensuring the query handles edge cases gracefully rather than throwing runtime errors.*

> *The Red Flag List applied dual filtering logic — WHERE for loan status at the row level before aggregation, and HAVING for DAR threshold at the group level after aggregation — demonstrating the precise distinction between row-level and group-level filtering in SQL.*

> *17 customers were identified as insolvent — DAR > 1 with overdue loan status. The most severe case showed a DAR of 39.51, meaning the borrower's outstanding debt is 39 times their available liquid assets. Total uncollateralized debt across all 17 red flag customers represents the bank's maximum potential loss exposure from this cohort.*

Phase 6 — CTE + Window Functions

▸ The Big Picture First

Domain context: Phase 4 told us the total portfolio numbers. Phase 5 identified insolvent individuals. Phase 6 asks a different question — **which types of customers default the most, and is default risk getting better or worse over time?** This requires a different analytical approach — not just aggregation, but **ranking, comparison, and trend analysis.** This is where CTEs and Window Functions become essential.

What Phase 6 does in one line:

> *We used Common Table Expressions to structure complex multi-step logic and Window Functions to rank default rates across customer segments and branches, and to analyze transaction trends over time — transforming raw loan data into a ranked risk intelligence system.*

▸ Window Functions Applied

Function	Applied To	Purpose
RANK()	Segments by default rate	Rank with gaps for ties
DENSE_RANK()	Segments by default rate	Rank without gaps
LAG()	Monthly transaction volume	Compare to previous month
LEAD()	Monthly transaction volume	Compare to next month
SUM() OVER (ORDER BY)	Loan exposure by start date	Running cumulative total

▸ Key Insight

> *CTE-based segment profiling transformed the 3-row customer_types reference table and 50,000-row loans table into a ranked default rate intelligence system. Large Enterprise customers showed the highest default rate at 11.02% — counterintuitively exceeding both Small Business (9.47%) and Individual (9.17%) segments — indicating that enterprise-level lending carries disproportionate concentration risk despite typically being considered lower risk.*

> *RANK() and DENSE_RANK() were applied simultaneously on segment default rates to demonstrate the distinction between gap-inclusive and gap-exclusive ranking. DENSE_RANK() is preferred for executive reporting where rank gaps create confusion.*

> *Branch-level default profiling required an indirect three-hop JOIN chain — branches → transactions → accounts → loans — because branches and loans share no direct foreign key relationship. This demonstrates the importance of understanding the full relational path before constructing analytical queries.*

> *LAG() and LEAD() window functions on monthly transaction volumes produced a month-over-month change metric — a leading indicator of portfolio health. Declining transaction activity in an account typically precedes formal loan default, making MoM_Change an early warning signal for proactive risk intervention.*

Phase 7 — Indexing & Performance Tuning

▸ The Big Picture First

Domain context: Everything built so far works correctly — but correctness is only half the requirement in a production banking system. The other half is **speed**. A risk report that takes 10 minutes to run is useless in a board meeting. A fraud detection query that takes 2 minutes to execute misses the fraud window. At 50,000 rows, your queries run fast. But real banking databases have 50 million transactions. The same query that runs in 136ms at 50,000 rows takes 2+ minutes at 50 million without indexes. Phase 7 is about proving your system is production-ready — not just analytically correct.

What Phase 7 does in one line:

> *We created B-Tree composite indexes on the highest-frequency filter and JOIN columns across four core tables, then used EXPLAIN and EXPLAIN ANALYZE to prove the difference — showing MySQL changing from full table scans to targeted index lookups, with real millisecond timing proof.*

▸ Indexes Created

Index Name	Table	Columns	Type
idx_loans_status_account	loans	LoanStatusID, AccountID	BTREE
idx_transactions_account	transactions_clean	AccountOriginID	BTREE
idx_customers_type	customers_final	CustomerTypeID	BTREE
idx_accounts_customer	accounts	CustomerID	BTREE

▸ Performance Results

- Before indexing: EXPLAIN showed type = ALL (full table scan) on loans table
- After indexing: type changed to ref — MySQL jumped directly to matching rows

- EXPLAIN ANALYZE on transactions_clean: actual execution time of 136ms across 50,000 rows
- Master portfolio query EXPLAIN confirmed type = ref on all indexed core tables
- INFORMATION_SCHEMA audit verified 4 B-Tree indexes registered across core tables

▸ Key Insight

> *Baseline EXPLAIN before indexing showed type = ALL on the loans table — confirming MySQL performed a full table scan examining all rows to satisfy a LoanStatusID filter. After creating the composite B-Tree index on (LoanStatusID, AccountID), the same query showed type = ref — MySQL now navigates directly to matching rows, eliminating the full scan entirely.*

> *EXPLAIN ANALYZE on the transactions_clean table returned an actual execution time of 136ms across 50,000 rows with a B-Tree index on AccountOriginID — demonstrating sub-second query performance at scale. At proportional scaling to 50 million rows with continued indexing, this architecture supports production-grade analytical workloads.*

> *INFORMATION_SCHEMA.STATISTICS audit confirmed 4 B-Tree indexes successfully registered across 4 core tables — customers_final (CustomerTypeID), loans (LoanStatusID, AccountID), transactions_clean (AccountOriginID), and accounts (CustomerID) — covering all high-frequency JOIN and WHERE clause columns used throughout the 57-query analytical pipeline.*

Phase 8a — Views & Stored Procedures

▸ The Big Picture First

Domain context: Everything built so far — the JOINs, the DAR analysis, the segment rankings — lives as raw SQL queries. Every time the risk team needs the Red Flag List, someone has to find the query, run it, and interpret it. That's not a system — that's a script. Phase 8a converts your analytical queries into a **permanent, reusable reporting layer** that any stakeholder can access with a simple SELECT statement, and automates the Credit Health Score calculation so the risk team can score any customer on demand without writing a single line of SQL.

What Phase 8a does in one line:

> *We created three permanent Views that abstract complex multi-table logic into simple queryable objects, and built a Stored Procedure that dynamically calculates a Credit Health Score for any customer based on their repayment history and transaction activity.*

▸ Views Created

View Name	Purpose	How to Query
vw_PortfolioHealth	Executive dashboard: total loans, principal, yield, % share by loan status	SELECT * FROM vw_PortfolioHealth

View Name	Purpose	How to Query
vw_RedFlagCustomers	Insolvent borrower list: DAR > 1, Overdue, uncollateralized debt ranked	SELECT * FROM vw_RedFlagCustomers
vw_SegmentRankings	Default rate % and RANK() per customer segment	SELECT * FROM vw_SegmentRankings

▸ Stored Procedure: sp_CreditHealthScore

Accepts CustomerID as input and returns a dynamic credit health score:

```
Score = (PaidOffLoans x 50) + (TotalTransactions x 0.1) - (OverdueLoans x 30)
CALL sp_CreditHealthScore(CustomerID);
```

▸ Key Insight

> *Three executive Views were created to abstract the complexity of 50,000 rows of raw data behind simple queryable objects — vw_PortfolioHealth for capital exposure and yield, vw_RedFlagCustomers for insolvent borrower detection, and vw_SegmentRankings for default rate intelligence. Non-technical stakeholders can retrieve all critical bank health metrics with a simple SELECT statement without any knowledge of the underlying multi-table JOIN logic.*

> *Views in MySQL are dynamic — they re-execute the underlying query on every access, ensuring results always reflect current data. This makes vw_RedFlagCustomers particularly powerful as a live risk monitoring tool — any change in loan status or account balance is automatically reflected in the next query without any manual refresh or maintenance.*

> *The sp_CreditHealthScore stored procedure implements a three-factor scoring model: repayment history (PaidOffLoans x 50), transaction activity (TotalTransactions x 0.1), and derogatory marks (OverdueLoans x 30 deducted). This mirrors the structure of real-world credit scoring models which combine positive behavioral signals with negative risk indicators into a single composite score.*

Phase 8b — What-If Interest Rate Simulation

▸ The Big Picture First

Domain context: A bank's profitability is directly tied to interest rates. When the central bank (like RBI in India or the Federal Reserve in the US) changes the base rate, every loan's yield changes with it. The bank's treasury team needs to answer: *"If rates drop by 1% tomorrow, how much revenue do we lose? If rates rise by 2%, how much do we gain? How does this affect our already-stressed overdue loans?"* This is called **interest rate stress testing** — and it's a regulatory requirement for all banks under Basel III norms.

What Phase 8b does in one line:

> *We built a complete interest rate stress testing system — from a static baseline through four rate shift scenarios to a dynamic stored procedure that simulates any rate change on*

> *demand — quantifying exact revenue impact and Expected Credit Loss changes across the entire loan portfolio.*

▸ Scenarios Simulated

Scenario	Rate Shift	Impact Measured
Rate increase (mild)	+1%	Expected yield gain across all loan statuses
Rate increase (severe)	+2%	Yield gain and ECL increase on overdue loans
Rate decrease (mild)	-1%	Expected yield loss and revenue delta
Rate decrease (severe)	-2%	Maximum revenue loss scenario

▸ Stored Procedure: sp_WhatIfSimulation

Accepts any rate shift as a DECIMAL input and returns simulated yield, current yield, and delta for the full portfolio.

```
CALL sp_WhatIfSimulation(0.01);   -- +1% rate increase
CALL sp_WhatIfSimulation(-0.02);  -- -2% rate decrease
```

▸ Key Insight

> *The What-If simulation engine quantified the bank's interest rate sensitivity — a +/-1% rate shift moves total portfolio yield by exactly $172,429, and a +/-2% shift by $344,858. The linear symmetry of these results confirms the mathematical correctness of the simulation model.*

> *ECL simulation on overdue loans revealed a counterintuitive finding — rising interest rates increase Expected Credit Loss on already-stressed borrowers. A +2% rate shift raises ECL from $136K to $169K, as higher rates compound the financial burden on overdue borrowers who are already failing to meet their obligations at current rates.*

> *The sp_WhatIfSimulation stored procedure elevates the simulation from a static report to a dynamic analytical tool — accepting any DECIMAL rate shift as input and returning full portfolio impact in real time. DECIMAL(5,4) was chosen over FLOAT to ensure exact arithmetic precision across 333 loan calculations.*

> *The complete Phase 8b system implements the core components of an IFRS 9 compliant interest rate risk framework, demonstrating that advanced SQL alone can power regulatory-grade financial risk analytics without requiring specialized software.*

5. RESULTS SUMMARY

Phase	Deliverable	Status
1 — Data Audit & Cleaning	customers_final (44 empty strings fixed), transactions_clean (500 duplicates removed)	Complete
2 — PK/FK Constraint Check	Zero referential integrity violations across all 6 relationships	Complete
3 — Multi-table JOINs	Master portfolio query unifying 6 tables in one result set	Complete
4 — Portfolio Quantization	Capital exposure, yield, % share scorecard by loan status and segment	Complete
5 — Liquidity Gap & DAR	Red Flag List of insolvent borrowers ranked by uncollateralized debt	Complete
6 — CTE + Window Functions	Segment default rate rankings, branch risk rank, MoM transaction trend	Complete
7 — Indexing & Performance	4 B-Tree indexes, EXPLAIN ANALYZE proof of 136ms execution time	Complete
8a — Views & Procedures	3 executive views + sp_CreditHealthScore stored procedure	Complete
8b — What-If Simulation	4-scenario rate stress test + sp_WhatIfSimulation stored procedure	Complete

6. LIMITATIONS & FUTURE SCOPE

6.1 Limitations

- The dataset is fully synthetic and does not reflect real borrower behavior, seasonal lending patterns, or macroeconomic cycles. All findings are analytically valid but cannot be extrapolated to real-world credit risk conclusions without validation against live banking data.
- The Credit Health Score uses a simplified three-factor formula. A production-grade scoring system would incorporate credit bureau data, employment history, collateral valuation, and time-series repayment patterns.
- The ECL simulation uses a simplified formula (PrincipalAmount x InterestRate) as a proxy. A full IFRS 9 compliant ECL model requires Probability of Default (PD), Loss Given Default (LGD), and Exposure at Default (EAD).
- The DAR analysis uses account balance as a proxy for liquid assets only. In a real banking context, liquid assets would also include fixed deposits, government securities, and other near-cash instruments.

6.2 Future Scope

- MySQL TRIGGERS for real-time Red Flag alerts — automatically flagging a customer the moment their loan status changes to Overdue and their DAR crosses 1.0.

- Integration with a Python or R analytics layer for machine learning-based credit scoring using the cleaned and aggregated SQL data as feature inputs.
- Power BI or Tableau dashboard connected directly to the MySQL Views for non-technical stakeholder monitoring without any SQL execution.
- Monte Carlo-style portfolio stress testing for Basel III Pillar 2 compliance — simulating simultaneous rate shifts combined with default rate changes.
- Full IFRS 9 ECL model incorporating historical default data with PD, LGD, and EAD components.

7. CONCLUSION

This project successfully transformed 50,000 rows of synthetic banking data into a fully operational SQL-based credit risk intelligence system. Starting from a raw, intentionally dirty dataset, the project delivered a complete pipeline from data cleaning through to executive reporting and interest rate stress testing.

The system demonstrates enterprise-grade SQL capabilities including multi-table JOIN chains, CTE-based segment profiling, window function analytics, B-Tree composite indexing, parameterized stored procedures, and dynamic financial simulation — all implemented in MySQL 8.0 on a real synthetic dataset.

Business Problems vs Solutions

Business Problem	Solution Delivered
Invisible Risk & Insolvent Borrowers	vw_RedFlagCustomers view + DAR-based Red Flag List with uncollateralized debt ranking
Data Integrity Gaps	Phase 1 cleaning pipeline — NULLIF, COALESCE, ROW_NUMBER() deduplication, date anomaly detection
Revenue Uncertainty	sp_WhatIfSimulation stored procedure — dynamic ECL and yield modeling across any rate scenario

8. REFERENCES

#	Reference
1	TestDataBox — Synthetic Finance Dataset (CC BY 4.0). Available at: kaggle.com
2	MySQL 8.0 Reference Manual — Window Functions. dev.mysql.com/doc
3	MySQL 8.0 Reference Manual — Stored Procedures. dev.mysql.com/doc
4	MySQL 8.0 Reference Manual — EXPLAIN Statement. dev.mysql.com/doc
5	Basel III: International Regulatory Framework for Banks. Bank for International Settlements, bis.org